

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

PART 1

```
CREATE DATABASE food_delivery_db;  
USE food_delivery_db;
```

```
CREATE TABLE customers (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  signup_date DATE  
);
```

```
CREATE TABLE restaurants (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  cuisine VARCHAR(50)  
);
```

```
CREATE TABLE orders (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  restaurant_id INT,  
  total DECIMAL(10,2),  
  status VARCHAR(30),  
  created_at DATETIME,
```

```
  FOREIGN KEY (customer_id)  
    REFERENCES customers(id),
```

```
  FOREIGN KEY (restaurant_id)  
    REFERENCES restaurants(id)
```

```
);
```

```
CREATE TABLE accounts (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  balance DECIMAL(10,2)  
);
```

INSERT INTO customers (name, email, signup_date) VALUES

('Neel Vaghela', 'neel@gmail.com', '2026-01-05'),
('Rahul Sharma', 'rahul@gmail.com', '2026-01-08'),
('Priya Patel', 'priya@gmail.com', '2026-01-10'),
('Amit Shah', 'amit@gmail.com', '2026-01-12'),
('Sneha Mehta', 'sneha@gmail.com', '2026-01-15'),
('Karan Joshi', 'karan@gmail.com', '2026-01-18'),
('Riya Desai', 'riya@gmail.com', '2026-01-20'),
('Arjun Singh', 'arjun@gmail.com', '2026-01-22'),
('Pooja Verma', 'pooja@gmail.com', '2026-01-25'),
('Vikas Kumar', 'vikas@gmail.com', '2026-01-28'),
('Anjali Gupta', 'anjali@gmail.com', '2026-02-01'),
('Harsh Patel', 'harsh@gmail.com', '2026-02-05'),
('Nisha Jain', 'nisha@gmail.com', '2026-02-08'),
('Rohan Mehta', 'rohan@gmail.com', '2026-02-12'),
('Divya Shah', 'divya@gmail.com', '2026-02-15'),
('Yash Patel', 'yash@gmail.com', '2026-02-18'),
('Krishna Patel', 'krishna@gmail.com', '2026-02-20'),
('Mansi Dave', 'manshi@gmail.com', '2026-02-22'),
('Jay Patel', 'jay@gmail.com', '2026-02-25'),
('Komal Shah', 'komal@gmail.com', '2026-02-28');

INSERT INTO restaurants (name, cuisine) VALUES

('Dominos', 'Pizza'),
('McDonalds', 'Fast Food'),
('KFC', 'Chicken'),
('Subway', 'Sandwich'),
('Pizza Hut', 'Pizza'),
('Barbeque Nation', 'BBQ'),
('Havmor', 'Ice Cream'),
('La Pinoz Pizza', 'Pizza'),
('The Grand Thakar', 'Gujarati'),
('Burger King', 'Burger');

INSERT INTO accounts (name, balance) VALUES

('Alice', 5000),
('Bob', 3000);

INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES

(1,1,450,'completed','2026-03-01 10:00:00'),
(2,2,320,'completed','2026-03-01 11:00:00'),
(3,3,600,'pending','2026-03-02 12:00:00'),
(4,4,250,'completed','2026-03-02 13:00:00'),
(5,5,780,'cancelled','2026-03-03 14:00:00'),
(6,6,900,'completed','2026-03-03 15:00:00'),
(7,7,150,'completed','2026-03-04 16:00:00'),
(8,8,450,'pending','2026-03-04 17:00:00'),
(9,9,700,'completed','2026-03-05 18:00:00'),
(10,10,350,'completed','2026-03-05 19:00:00'),
(11,1,420,'completed','2026-03-06 10:00:00'),
(12,2,550,'pending','2026-03-06 11:00:00'),

(13,3,660,'completed','2026-03-07 12:00:00'),
(14,4,340,'completed','2026-03-07 13:00:00'),
(15,5,870,'cancelled','2026-03-08 14:00:00'),
(16,6,920,'completed','2026-03-08 15:00:00'),
(17,7,180,'completed','2026-03-09 16:00:00'),
(1,8,510,'completed','2026-03-09 17:00:00'),
(2,9,730,'pending','2026-03-10 18:00:00'),
(3,10,390,'completed','2026-03-10 19:00:00'),
(4,1,460,'completed','2026-03-11 10:00:00'),
(5,2,330,'completed','2026-03-11 11:00:00'),
(6,3,610,'pending','2026-03-12 12:00:00'),
(7,4,260,'completed','2026-03-12 13:00:00'),
(8,5,790,'cancelled','2026-03-13 14:00:00'),
(9,6,910,'completed','2026-03-13 15:00:00'),
(10,7,160,'completed','2026-03-14 16:00:00'),
(11,8,470,'pending','2026-03-14 17:00:00'),
(12,9,710,'completed','2026-03-15 18:00:00'),
(13,10,360,'completed','2026-03-15 19:00:00'),
(14,1,430,'completed','2026-03-16 10:00:00'),
(15,2,540,'pending','2026-03-16 11:00:00'),
(16,3,650,'completed','2026-03-17 12:00:00'),
(17,4,300,'completed','2026-03-17 13:00:00'),
(1,5,820,'cancelled','2026-03-18 14:00:00'),
(2,6,950,'completed','2026-03-18 15:00:00'),
(3,7,170,'completed','2026-03-19 16:00:00'),
(4,8,520,'pending','2026-03-19 17:00:00'),
(5,9,740,'completed','2026-03-20 18:00:00'),
(6,10,370,'completed','2026-03-20 19:00:00'),
(7,1,440,'completed','2026-03-21 10:00:00'),
(8,2,560,'pending','2026-03-21 11:00:00'),
(9,3,670,'completed','2026-03-22 12:00:00'),
(10,4,280,'completed','2026-03-22 13:00:00'),
(11,5,800,'cancelled','2026-03-23 14:00:00'),
(12,6,930,'completed','2026-03-23 15:00:00'),
(13,7,190,'completed','2026-03-24 16:00:00'),
(14,8,480,'pending','2026-03-24 17:00:00'),
(15,9,720,'completed','2026-03-25 18:00:00'),
(16,10,380,'completed','2026-03-25 19:00:00');

PART 2

TASK 1 Transactions (ACID)

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

```
1 START TRANSACTION;
2 UPDATE accounts
3 SET balance = balance - 200
4 WHERE id = 1;
5 UPDATE accounts
6 SET balance = balance + 200
7 WHERE id = 2; SELECT * FROM accounts;
```

Result Grid

id	name	balance
1	Alice	4800.00
2	Bob	3200.00

accounts 21

Output

Action Output

#	Time	Action	Message	Duration / Fetch
31	16:27:20	SELECT COUNT(*) FROM accounts LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
32	17:54:43	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.016 sec / 0.000 sec
33	17:58:45	START TRANSACTION	0 row(s) affected	0.000 sec
34	17:58:45	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
35	17:58:45	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
36	17:58:45	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

```
1 ROLLBACK;
2 SELECT * FROM accounts;
```

Result Grid

id	name	balance
1	Alice	5000.00
2	Bob	3000.00

accounts 22

Output

Action Output

#	Time	Action	Message	Duration / Fetch
33	17:58:45	START TRANSACTION	0 row(s) affected	0.000 sec
34	17:58:45	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
35	17:58:45	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
36	17:58:45	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
37	18:00:36	ROLLBACK	0 row(s) affected	0.015 sec
38	18:00:36	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

```

1 • START TRANSACTION;
2 • UPDATE accounts
3   SET balance = balance - 200
4   WHERE id = 1;
5 • UPDATE accounts
6   SET balance = balance + 200
7   WHERE id = 2;

```

Limit to 1000 rows

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

id	name	balance
1	Alice	4200.00
2	Bob	3600.00

accounts 23

Output

Action Output

#	Time	Action	Message	Duration / Fetch
47	18:08:06	COMMIT	0 row(s) affected	0.000 sec
48	18:08:18	START TRANSACTION	0 row(s) affected	0.000 sec
49	18:08:18	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
50	18:08:18	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
51	18:08:18	COMMIT	0 row(s) affected	0.031 sec
52	18:08:18	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

```

3   SET balance = balance - 200
4   WHERE id = 1;
5 • UPDATE accounts
6   SET balance = balance + 200
7   WHERE id = 2;
8 • COMMIT;
9 • SELECT * FROM accounts;

```

Limit to 1000 rows

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

id	name	balance
1	Alice	4200.00
2	Bob	3600.00

accounts 23

Output

Action Output

#	Time	Action	Message	Duration / Fetch
47	18:08:06	COMMIT	0 row(s) affected	0.000 sec
48	18:08:18	START TRANSACTION	0 row(s) affected	0.000 sec
49	18:08:18	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
50	18:08:18	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
51	18:08:18	COMMIT	0 row(s) affected	0.031 sec
52	18:08:18	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Atomicity is the ACID property that ensures a transaction is completed entirely or not at all. During a money transfer, both the debit and credit operations are treated as a single unit. If any part of the transaction fails, the entire transaction is rolled back, ensuring that no money is lost or partially transferred.

TASK 2 Joins

a) Write an INNER JOIN that lists every customer who has placed at least one order.

The screenshot shows the MySQL Workbench interface with a query window titled 'Query 1'. The query is an INNER JOIN between the 'customers' and 'orders' tables. The results are displayed in a grid with columns: id, name, email, order_id, total, and status. The results show 18 rows of data.

```
1 SELECT c.id,c.name,c.email,o.id AS order_id,o.total,o.status
2 FROM customers c
3 INNER JOIN orders o
4 ON c.id = o.customer_id;
```

id	name	email	order_id	total	status
1	Neel Vaghela	neel@gmail.com	1	450.00	completed
2	Rahul Sharma	rahul@gmail.com	2	320.00	completed
3	Priya Patel	priya@gmail.com	3	600.00	pending
4	Amit Shah	amit@gmail.com	4	250.00	completed
5	Sneha Mehta	sneha@gmail.com	5	780.00	cancelled
6	Karan Joshi	karan@gmail.com	6	900.00	completed
7	Riya Desai	riya@gmail.com	7	150.00	completed
8	Arjun Singh	arjun@gmail.com	8	450.00	pending
9	Pooja Verma	pooja@gmail.com	9	700.00	completed
10	Vikas Kumar	vikas@gmail.com	10	350.00	completed
11	Anjali Gupta	anjali@gmail.com	11	420.00	completed
12	Harsh Patel	harsh@gmail.com	12	550.00	pending
13	Nisha Jain	nisha@gmail.com	13	660.00	completed
14	Rohan Mehta	rohan@gmail.com	14	340.00	completed
15	Divya Shah	divya@gmail.com	15	870.00	cancelled
16	Yash Patel	yash@gmail.com	16	920.00	completed
17	Krishna Patel	krishna@gmail.com	17	180.00	completed
18	Neel Vaghela	neel@gmail.com	18	510.00	completed

The output window shows the execution of the query, including the time taken and the number of rows returned.

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

The screenshot shows the MySQL Workbench interface with a query window titled 'Query 1'. The query is a LEFT JOIN between the 'customers' and 'orders' tables, with a WHERE clause to filter out customers who have never ordered. The results are displayed in a grid with columns: id, name, email, order_id, total, and status. The results show 3 rows of data.

```
1 SELECT c.id,c.name,c.email
2 FROM customers c
3 LEFT JOIN orders o
4 ON c.id = o.customer_id
5 WHERE o.id IS NULL;
```

id	name	email	order_id	total	status
18	Mansi Dave	mansi@gmail.com			
19	Jay Patel	jay@gmail.com			
20	Komal Shah	komal@gmail.com			

The output window shows the execution of the query, including the time taken and the number of rows returned.

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists databases: food_delivery_db, studentdb, and sys. The 'Query' editor in the center contains the following SQL query:

```

1 SELECT c.id,c.name,o.id AS order_id,o.total
2 FROM customers c
3 LEFT JOIN orders o
4 ON c.id = o.customer_id;

```

The 'Result Grid' displays the query results with columns: id, name, order_id, and total. The results are as follows:

id	name	order_id	total
1	Neel Vaghela	1	450.00
1	Neel Vaghela	18	510.00
1	Neel Vaghela	35	820.00
2	Rahul Sharma	2	320.00
2	Rahul Sharma	19	730.00
2	Rahul Sharma	36	950.00
3	Priya Patel	3	600.00
3	Priya Patel	20	390.00
3	Priya Patel	37	170.00
4	Amit Shah	4	250.00
4	Amit Shah	21	460.00
4	Amit Shah	38	320.00
5	Sneha Mehta	5	780.00
5	Sneha Mehta	22	330.00
5	Sneha Mehta	39	740.00
6	Karan Joshi	6	900.00
6	Karan Joshi	23	610.00
6	Karan Joshi	40	370.00

The 'Output' pane at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
2	17:20:17	USE food_delivery_db	0 row(s) affected	0.000 sec
3	17:20:37	SELECT c.id, c.name, c.email, o.id AS order_id, o.total, o.status FROM customers c INNER JOIN orders o ON c.id = o.customer_id	50 row(s) returned	0.031 sec / 0.000 sec
4	17:23:55	SELECT c.id,c.name,o.id AS order_id,o.total FROM customers c LEFT JOIN orders o ON c.id = o.customer_id	53 row(s) returned	0.000 sec / 0.000 sec

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

A marketing team would prefer the LEFT JOIN result because it includes customers who have never placed an order, allowing the company to target them with promotional offers, discounts, or reminder campaigns.

TASK 3 Indexing

- Pick a column that is not currently indexed (for example status or created_at).
- Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains the following SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'completed';
```

The 'Result Grid' shows the EXPLAIN output for the query. The first row indicates a table scan on the 'orders' table.

#	Time	Action	Message	Duration / Fetch
4	17:23:55	SELECT c.id,c.name,o.id AS order_id,o.total FROM customers c LEFT JOIN orders o ON c.id = o.customer_id ...	53 row(s) returned	0.000 sec / 0.000 sec
5	17:25:30	SELECT c.id,c.name,c.email FROM customers c LEFT JOIN orders o ON c.id = o.customer_id WHERE o.id IS ...	3 row(s) returned	0.000 sec / 0.000 sec
6	17:37:08	EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'completed'	1 row(s) returned	0.016 sec / 0.000 sec

- Create an index on that column.

The screenshot shows the MySQL Workbench interface after creating an index. The 'Query 1' window contains the following SQL queries:

```
1 CREATE INDEX idx_status
2 ON orders(status);
3 SHOW INDEX FROM orders;
```

The 'Result Grid' shows the output of the 'SHOW INDEX FROM orders' query, indicating that the index 'idx_status' has been created on the 'status' column.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	id	A	50				BTREE			YES	
orders	1	customer_id	1	customer_id	A	17			YES	BTREE			YES	
orders	1	restaurant_id	1	restaurant_id	A	10			YES	BTREE			YES	
orders	1	idx_status	1	status	A	3			YES	BTREE			YES	

The 'Result 5' window shows the output of the 'EXPLAIN ANALYZE' query after the index was created. The first row indicates that the index 'idx_status' was used.

#	Time	Action	Message	Duration / Fetch
6	17:37:08	EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'completed'	1 row(s) returned	0.016 sec / 0.000 sec
7	17:39:52	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec
8	17:39:52	SHOW INDEX FROM orders	4 row(s) returned	0.015 sec / 0.000 sec

d) Re-run the exact same query and record the new numbers.

The screenshot shows the MySQL Workbench interface. The 'Query 1' tab contains the following SQL code:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'completed';
```

The 'Result Grid' shows the output of the EXPLAIN command:

EXPLAIN
--> Index lookup on orders using idx_status (status='completed') (cost=4.25 rows=35) (actual time=0.145..0.186 rows=35 loops=1)

The 'Output' tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
7	17:39:52	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec
8	17:39:52	SHOW INDEX FROM orders	4 row(s) returned	0.015 sec / 0.000 sec
9	17:40:44	EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'completed'	1 row(s) returned	0.000 sec / 0.000 sec

e) Submit a one-line comparison of before vs. after.

After creating the index on the status column, MySQL used the `idx_status` index to locate matching rows more efficiently, reducing the amount of work required and improving query performance.

TASK 4 Query Optimization

a) Rewrite it as a JOIN instead of a subquery.

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Administration Schemas

Information

No object selected

Object Info Session

Query 1

Limit to 1000 rows

1 SELECT *

2 FROM orders

3 WHERE customer_id IN (

4 SELECT id

5 FROM customers

6 WHERE signup_date >= '2026-02-01'

7);

Result Grid

Filter Rows:

id customer_id restaurant_id total status created_at

11 11 1 420.00 completed 2026-03-06 10:00:00

28 11 8 470.00 pending 2026-03-14 17:00:00

45 11 5 800.00 cancelled 2026-03-23 14:00:00

12 12 2 550.00 pending 2026-03-06 11:00:00

29 12 9 710.00 completed 2026-03-15 18:00:00

46 12 6 930.00 completed 2026-03-23 15:00:00

13 13 3 660.00 completed 2026-03-07 12:00:00

30 13 10 360.00 completed 2026-03-15 19:00:00

47 13 7 190.00 completed 2026-03-24 16:00:00

14 14 4 340.00 completed 2026-03-07 13:00:00

31 14 1 430.00 completed 2026-03-16 10:00:00

48 14 8 480.00 pending 2026-03-24 17:00:00

15 15 5 870.00 cancelled 2026-03-08 14:00:00

32 15 2 540.00 pending 2026-03-16 11:00:00

49 15 9 720.00 completed 2026-03-25 18:00:00

16 16 6 920.00 completed 2026-03-08 15:00:00

33 16 3 650.00 completed 2026-03-17 12:00:00

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

orders 7 x

Output

Action Output

Time Action Message Duration / Fetch

9 17:40:44 EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'completed' 1 row(s) returned 0.000 sec / 0.000 sec

10 17:46:43 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date >= '2026-02-01') 20 row(s) returned 0.000 sec / 0.000 sec

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Administration Schemas

Information

No object selected

Object Info Session

Query 1

Limit to 1000 rows

1 SELECT

2 o.*

3 FROM orders o

4 INNER JOIN customers c

5 ON o.customer_id = c.id

6 WHERE c.signup_date >= '2026-02-01';

Result Grid

Filter Rows:

id customer_id restaurant_id total status created_at

11 11 1 420.00 completed 2026-03-06 10:00:00

28 11 8 470.00 pending 2026-03-14 17:00:00

45 11 5 800.00 cancelled 2026-03-23 14:00:00

12 12 2 550.00 pending 2026-03-06 11:00:00

29 12 9 710.00 completed 2026-03-15 18:00:00

46 12 6 930.00 completed 2026-03-23 15:00:00

13 13 3 660.00 completed 2026-03-07 12:00:00

30 13 10 360.00 completed 2026-03-15 19:00:00

47 13 7 190.00 completed 2026-03-24 16:00:00

14 14 4 340.00 completed 2026-03-07 13:00:00

31 14 1 430.00 completed 2026-03-16 10:00:00

48 14 8 480.00 pending 2026-03-24 17:00:00

15 15 5 870.00 cancelled 2026-03-08 14:00:00

32 15 2 540.00 pending 2026-03-16 11:00:00

49 15 9 720.00 completed 2026-03-25 18:00:00

16 16 6 920.00 completed 2026-03-08 15:00:00

33 16 3 650.00 completed 2026-03-17 12:00:00

50 16 10 380.00 completed 2026-03-25 19:00:00

17 17 7 180.00 completed 2026-03-09 16:00:00

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result 8 x

Output

Action Output

Time Action Message Duration / Fetch

10 17:46:43 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date >= '2026-02-01') 20 row(s) returned 0.000 sec / 0.000 sec

11 17:48:31 SELECT o.* FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2026-02-01' 20 row(s) returned 0.000 sec / 0.000 sec

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

Limit to 1000 rows

```

1 SELECT o.id AS order_id,c.name AS customer_name,o.total,o.status,o.created_at
2 FROM orders o
3 INNER JOIN customers c
4 ON o.customer_id = c.id
5 WHERE c.signup_date >= '2026-02-01';

```

Result Grid

order_id	customer_name	total	status	created_at
11	Anjali Gupta	420.00	completed	2026-03-06 10:00:00
28	Anjali Gupta	470.00	pending	2026-03-14 17:00:00
45	Anjali Gupta	800.00	cancelled	2026-03-23 14:00:00
12	Harsh Patel	550.00	pending	2026-03-06 11:00:00
29	Harsh Patel	710.00	completed	2026-03-15 18:00:00
46	Harsh Patel	930.00	completed	2026-03-23 15:00:00
13	Nisha Jain	660.00	completed	2026-03-07 12:00:00
30	Nisha Jain	360.00	completed	2026-03-15 19:00:00
47	Nisha Jain	190.00	completed	2026-03-24 16:00:00
14	Rohan Mehta	340.00	completed	2026-03-07 13:00:00
31	Rohan Mehta	430.00	completed	2026-03-16 10:00:00
48	Rohan Mehta	480.00	pending	2026-03-24 17:00:00
15	Divya Shah	870.00	cancelled	2026-03-08 14:00:00
32	Divya Shah	540.00	pending	2026-03-16 11:00:00
49	Divya Shah	720.00	completed	2026-03-25 18:00:00
16	Yash Patel	920.00	completed	2026-03-08 15:00:00
33	Yash Patel	650.00	completed	2026-03-17 12:00:00
50	Yash Patel	380.00	completed	2026-03-25 19:00:00
17	Krishna Patel	180.00	completed	2026-03-09 16:00:00
34	Krishna Patel	300.00	completed	2026-03-17 13:00:00

Result 9

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
11	17:48:31	SELECT o.* FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2026-02-01';	20 row(s) returned	0.000 sec / 0.000 sec
12	17:49:59	SELECT o.id AS order_id,c.name AS customer_name,o.total,o.status,o.created_at FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2026-02-01';	20 row(s) returned	0.015 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

Limit to 1000 rows

```

1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id IN (
5   SELECT id
6   FROM customers
7   WHERE signup_date >= '2026-02-01'
8 );

```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Filter Rows: Exports Wrap Cell Content: 12

EXPLAIN

-> Nested loop inner join (cost=9.11 rows=19.6) (actual time=3.65..3.87 rows=20 loops=1) -> Filter: (customers.signup_date >= DATE'2026-02-01') (cost=2.25 rows=6.67) (actual time=3.48..3.5 rows=10 loops=1) -> Table scan on customers (cost=...)

Administration Schemas

Information

No object selected

Result 13

Output

Action Output

#	Time	Action	Message	Duration / Fetch
15	18:00:02	EXPLAIN ANALYZE SELECT o.id,c.name,o.total,o.status,o.created_at FROM orders o INNER JOIN customer...	1 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

MySQL Workbench

Restaurant

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

food_delivery_db

Tables

Views

Stored Procedures

Functions

studentdb

sys

Query 1

Limit to 1000 rows

```

1 EXPLAIN ANALYZE
2 SELECT o.id,c.name,o.total,o.status,o.created_at
3 FROM orders o
4 INNER JOIN customers c
5 ON o.customer_id = c.id
6 WHERE c.signup_date >= '2026-02-01';

```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Filter Rows: Exports Wrap Cell Content: 12

EXPLAIN

-> Nested loop inner join (cost=9.11 rows=19.6) (actual time=0.157..0.277 rows=20 loops=1) -> Filter: (c.signup_date >= DATE'2026-02-01') (cost=2.25 rows=6.67) (actual time=0.0954..0.108 rows=10 loops=1) -> Table scan on c (cost=2.25 rows=...)

Administration Schemas

Information

No object selected

Result 12

Output

Action Output

#	Time	Action	Message	Duration / Fetch
14	17:59:49	EXPLAIN SELECT o.id,c.name,o.total,o.status,o.created_at FROM orders o INNER JOIN customers c ON o.c...	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

PART 3

TASK 5 The Customer Profile Page

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view containing 'food_delivery_db', 'studentdb', and 'sys'. The main query editor shows a SQL query that selects restaurant name, total, status, and created_at from the orders table, joined with the restaurants table, filtered by customer_id = 1, and sorted by created_at in descending order, limited to 5 results.

```
1 SELECT
2   r.name AS restaurant_name,
3   o.total,
4   o.status,
5   o.created_at
6 FROM orders o
7 INNER JOIN restaurants r
8 ON o.restaurant_id = r.id
9 WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
```

The 'Result Grid' shows the following data:

restaurant_name	total	status	created_at
Pizza Hut	820.00	cancelled	2026-03-18 14:00:00
La Pinoz Pizza	510.00	completed	2026-03-09 17:00:00
Dominos	450.00	completed	2026-03-01 10:00:00

The bottom panel shows the 'Output' tab with a table of actions and their durations:

#	Time	Action	Message	Duration / Fetch
14	17:59:49	EXPLAIN SELECT o.id,c.name,o.total,o.status,o.created_at FROM orders o INNER JOIN customers c ON o.c...	2 row(s) returned	0.000 sec / 0.000 sec
15	18:00:02	EXPLAIN ANALYZE SELECT o.id,c.name,o.total,o.status,o.created_at FROM orders o INNER JOIN customer...	1 row(s) returned	0.016 sec / 0.000 sec
16	18:20:42	EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers ...	1 row(s) returned	0.031 sec / 0.000 sec
17	18:31:16	SELECT r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o INNER JOIN re...	3 row(s) returned	0.015 sec / 0.000 sec

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL:

```

1  EXPLAIN ANALYZE
2  SELECT
3      r.name AS restaurant_name,
4      o.total,
5      o.status,
6      o.created_at
7  FROM orders o
8  INNER JOIN restaurants r
9  ON o.restaurant_id = r.id
10 WHERE o.customer_id = 1
11 ORDER BY o.created_at DESC
12 LIMIT 5;

```

The 'Result Grid' shows the execution plan for the query. The plan indicates a nested loop inner join with a cost of 2.1 rows=3. The execution time is 0.151 seconds. The plan also shows the sort operation for the ORDER BY clause.

The 'Output' tab shows the execution history. The query was executed at 18:00:02, 18:20:42, 18:31:16, and 18:33:08. The execution time for the query was 0.016 seconds, 0.031 seconds, 0.015 seconds, and 0.031 seconds respectively.

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL:

```

1  CREATE INDEX idx_customer_created
2  ON orders(customer_id, created_at);
3  SHOW INDEX FROM orders;

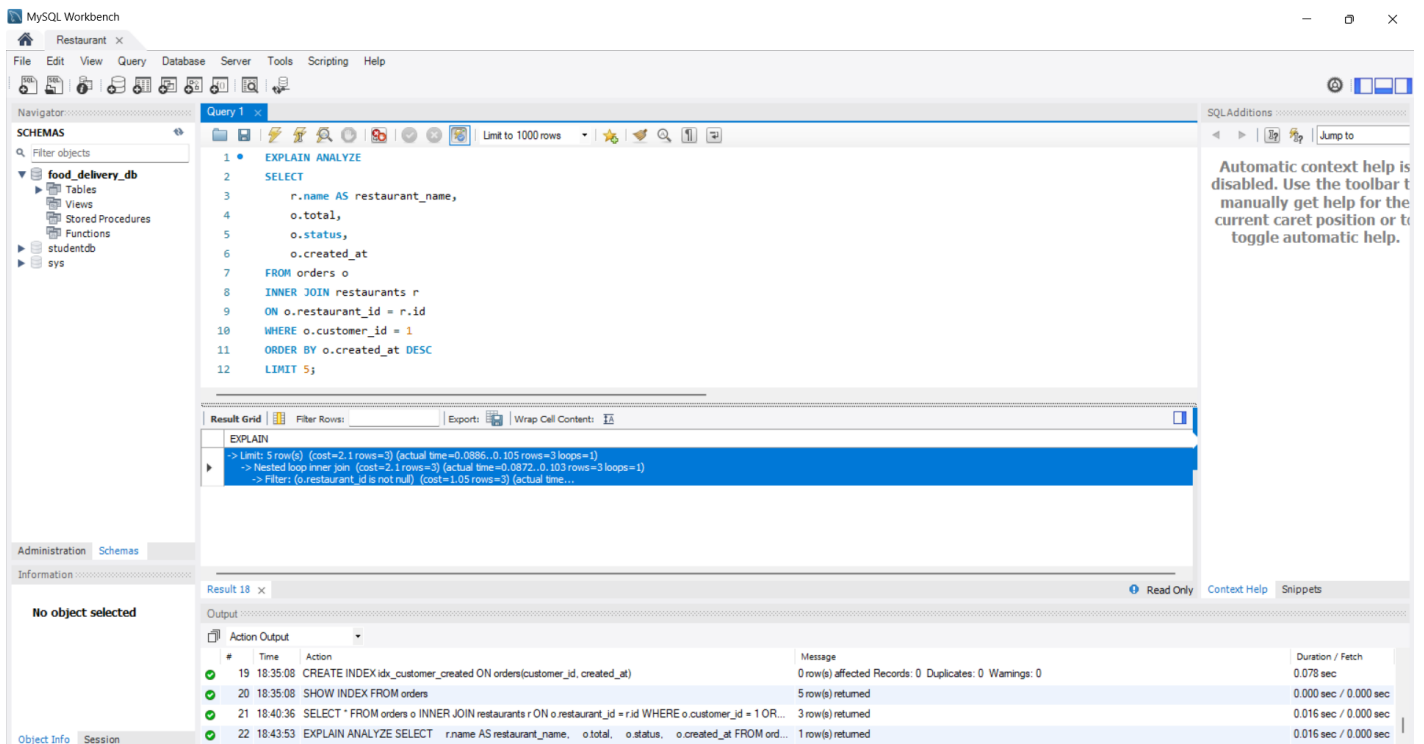
```

The 'Result Grid' shows the execution plan for the query. The plan indicates a nested loop inner join with a cost of 2.1 rows=3. The execution time is 0.151 seconds. The plan also shows the sort operation for the ORDER BY clause.

The 'Output' tab shows the execution history. The query was executed at 18:00:02, 18:20:42, 18:31:16, and 18:33:08. The execution time for the query was 0.016 seconds, 0.031 seconds, 0.015 seconds, and 0.031 seconds respectively.

The 'Result Grid' shows the execution plan for the query. The plan indicates a nested loop inner join with a cost of 2.1 rows=3. The execution time is 0.151 seconds. The plan also shows the sort operation for the ORDER BY clause.

d) Re-run the same query and compare the new numbers to step (b).



MySQL Workbench interface showing the execution of a query. The query is as follows:

```

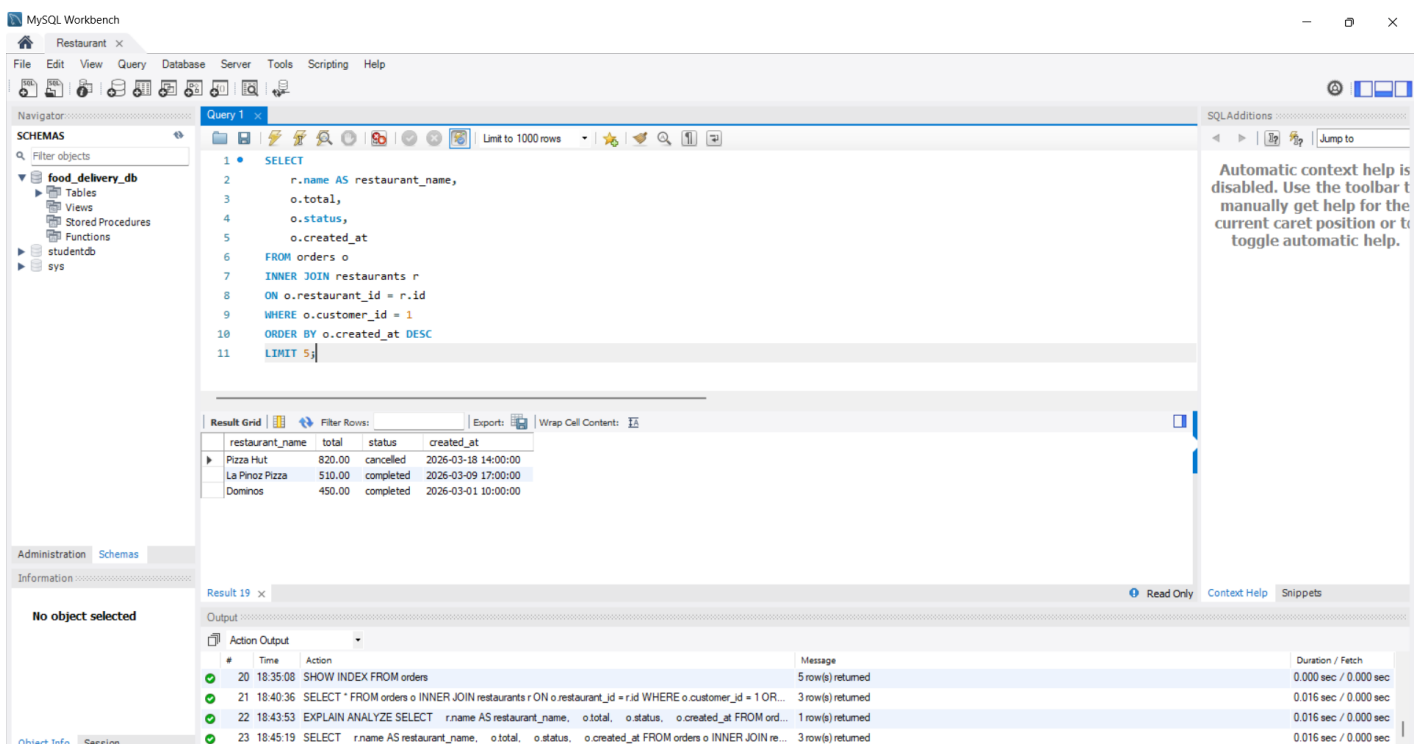
1  EXPLAIN ANALYZE
2  SELECT
3      r.name AS restaurant_name,
4      o.total,
5      o.status,
6      o.created_at
7  FROM orders o
8  INNER JOIN restaurants r
9  ON o.restaurant_id = r.id
10 WHERE o.customer_id = 1
11 ORDER BY o.created_at DESC
12 LIMIT 5;

```

The output shows the execution plan and the results of the query. The execution plan indicates that the query is executed using a nested loop inner join. The results show the restaurant details for the customer with ID 1, ordered by the creation time of the order.

#	Time	Action	Message	Duration / Fetch
19	18:35:08	CREATE INDEX idx_customer_created ON orders(customer_id, created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec
20	18:35:08	SHOW INDEX FROM orders	5 row(s) returned	0.000 sec / 0.000 sec
21	18:40:36	SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 1 OR...	3 row(s) returned	0.016 sec / 0.000 sec
22	18:43:53	EXPLAIN ANALYZE SELECT r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 1 ORDER BY o.created_at DESC LIMIT 5;	1 row(s) returned	0.016 sec / 0.000 sec

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.



MySQL Workbench interface showing the execution of a query. The query is as follows:

```

1  SELECT
2      r.name AS restaurant_name,
3      o.total,
4      o.status,
5      o.created_at
6  FROM orders o
7  INNER JOIN restaurants r
8  ON o.restaurant_id = r.id
9  WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
11 LIMIT 5;

```

The output shows the execution plan and the results of the query. The execution plan indicates that the query is executed using a nested loop inner join. The results show the restaurant details for the customer with ID 1, ordered by the creation time of the order.

restaurant_name	total	status	created_at
Pizza Hut	820.00	cancelled	2026-03-18 14:00:00
La Pinoz Pizza	510.00	completed	2026-03-09 17:00:00
Dominos	450.00	completed	2026-03-01 10:00:00

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Before indexing: MySQL had to examine more rows to find the required customer's orders and then sort them by the order date.

After indexing: MySQL used the `idx_customer_created` index to quickly locate the customer's orders and retrieve them in the required order, reducing the amount of work needed.

The composite index on `(customer_id, created_at)` is effective because `customer_id` is used in the `WHERE` clause to filter the records, and `created_at` is used in the `ORDER BY` clause to sort them. This allows MySQL to retrieve the latest orders more efficiently.